

Here is the exact code and step-by-step additions needed to bridge the backend to your HemaLink management portals, covering the missing **Partner Inventory**, **Hospital Verification**, and **Institutional Multi-Unit Fulfillment**.

Step 1: Create the Partner Inventory Module

Create a new directory inside the backend: BloodBridge/backend/src/inventory/.

src/inventory/models.py

```
Python
import uuid
from datetime import datetime
from sqlalchemy import Column, String, Integer, DateTime, ForeignKey
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import relationship
from src.utils.database import Base

class BloodInventory(Base):
    tablename = "blood_inventories"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    organization_id = Column(UUID(as_uuid=True), ForeignKey("organizations.id"),
        nullable=False, index=True)
    blood_group = Column(String(5), nullable=False, # e.g. "A+", "O-", etc.
    component_type = Column(String(50), nullable=False, # "Whole Blood", "PRBC", "Platelets",
"FFP"
    units_available = Column(Integer, nullable=False, default=0)
    last_updated = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    organization = relationship("Organization", backref="inventories")
```

src/inventory/dtos.py

```
Python
from pydantic import BaseModel, Field
from uuid import UUID
from datetime import datetime
```

```

class InventoryUpdateDTO(BaseModel):
    blood_group: str
    component_type: str = Field(default="Whole Blood")
    units: int = Field(ge=0)

class InventoryResponseDTO(BaseModel):
    id: UUID
    organization_id: UUID
    blood_group: str
    component_type: str
    units_available: int
    last_updated: datetime

class Config:
    from_attributes = True

```

src/inventory/router.py

```

Python
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from uuid import UUID
from typing import List

from src.utils.database import get_db
from src.utils.auth import get_current_user
from src.inventory.models import BloodInventory
from src.inventory.dtos import InventoryUpdateDTO, InventoryResponseDTO

router = APIRouter(prefix="/inventory", tags=["Partner Inventory"])

@router.get("", response_model=List[InventoryResponseDTO])
def get_inventory(
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
    # Only partner institutions can view their own stock
    if current_user.get("role") != "partner":
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Only partner

```

```
organizations can view inventory"
```

```
org_id = current_user.get("organization_id")
return db.query(BloodInventory).filter(BloodInventory.organization_id == org_id).all()
```

```
@router.post("/update", response_model=InventoryResponseDTO)
```

```
def update_stock(
    data: InventoryUpdateDTO,
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
    if current_user.get("role") != "partner":
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Only partner
organizations can update inventory")
```

```
org_id = current_user.get("organization_id")
item = db.query(BloodInventory).filter(
    BloodInventory.organization_id == org_id,
    BloodInventory.blood_group == data.blood_group,
    BloodInventory.component_type == data.component_type
).first()
```

```
if not item:
    item = BloodInventory(
        organization_id=org_id,
        blood_group=data.blood_group,
        component_type=data.component_type,
        units_available=data.units
    )
    db.add(item)
else:
    item.units_available = data.units
```

```
db.commit()
db.refresh(item)
return item
```

Step 2: Add Hospital Verification & Partner Fulfillment Endpoints

Add these two specialized management routes into
BloodBridge/backend/src/blood_requests/router.py:

Python

```
from pydantic import BaseModel, Field
from src.inventory.models import BloodInventory
```

```
class FulfillRequestDTO(BaseModel):
    units_to_fulfill: int = Field(gt=0)
    component_type: str = "Whole Blood"
```

```
@router.post("/{request_id}/verify")
def verify_request_by_hospital(
    request_id: UUID,
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
    # Enforce Rule: Only hospital role can verify
    if current_user.get("role") != "hospital":
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Only hospital staff can verify patient requests.")
```

```
    hospital_id = current_user.get("hospital_id")
    blood_req = db.query(BloodRequest).filter(BloodRequest.id == request_id).first()
```

```
    if not blood_req:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Blood request not found.")
```

```
    # Enforce Rule: Hospital can ONLY verify requests for their registered facility
    if str(blood_req.hospital_id) != str(hospital_id):
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Cannot verify requests for another hospital facility.")
```

```
    blood_req.status = "VERIFIED"
    db.commit()
    return "message" "Request verified successfully" "status" blood_req.status
```

```
@router.post("/{request_id}/fulfill")
def fulfill_request_by_partner(
    request_id: UUID,
    payload: FulfillRequestDTO,
    db: Session = Depends(get_db),
    current_user: dict = Depends(get_current_user)
):
```

```

# Enforce Rule: Only partner organizations fulfill requests from stock
if current_user.get("role") != "partner":
    raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Only partner
institutions can fulfill external requests.")

org_id = current_user.get("organization_id")
blood_req = db.query(BloodRequest).filter(BloodRequest.id == request.id).first()

if not blood_req:
    raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Blood
request not found.")

if blood_req.status not in "VERIFIED" "PARTIALLY_FULFILLED":
    raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Only
verified active requests can be fulfilled.")

# Check partner cold storage stock
stock = db.query(BloodInventory).filter(
    BloodInventory.organization_id == org_id
    BloodInventory.blood_group == blood_req.blood_group
    BloodInventory.component_type == payload.component_type
).first()

if not stock or stock.units_available < payload.units_to_fulfill:
    raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Insufficient
blood units in inventory.")

# Deduct stock and update request units
stock.units_available -= payload.units_to_fulfill
blood_req.units_fulfilled = (blood_req.units_fulfilled or 0) + payload.units_to_fulfill

# Enforce Rule: Fulfilled only when all required units are completed
if blood_req.units_fulfilled >= blood_req.units_required:
    blood_req.status = "FULFILLED"
else:
    blood_req.status = "PARTIALLY_FULFILLED"

db.commit()
return {
    "message": f"Successfully fulfilled {payload.units_to_fulfill} unit(s).",
    "units_remaining": max(0, blood_req.units_required - blood_req.units_fulfilled),
    "status": blood_req.status,
}

```

Step 3: Register the Inventory Router in main.py

Open BloodBridge/backend/main.py and register the new inventory router:

Python

```
from src.inventory.router import router as inventory_router
```

```
# Attach along with the existing routers
```

```
app.include_router(inventory_router, prefix="/api/v1")
```

Step 4: Apply Database Migrations

Generate and run an Alembic migration so PostgreSQL creates the blood_inventories table and adds units_fulfilled to blood_requests:

Bash

```
cd bloodBridge/backend
```

```
alembic revision --autogenerate -m "add_inventory_and_fulfillment_fields"
```

```
alembic upgrade head
```

Once this is run, your backend will support the exact state transitions and permissions required by your React portal dashboards.